B103 Databases & Big Data

Individual Project Report

Gisma University of Applied Sciences

Student Name: Ravshanjon Rakhmonberdiev

Student ID: GH1047114

GitHub Repository: https://github.com/ravshanjohn0

Video Demonstration: **database_explanation.mp4 - OneDrive**
DB Diagram: **uni - dbdiagram.io**

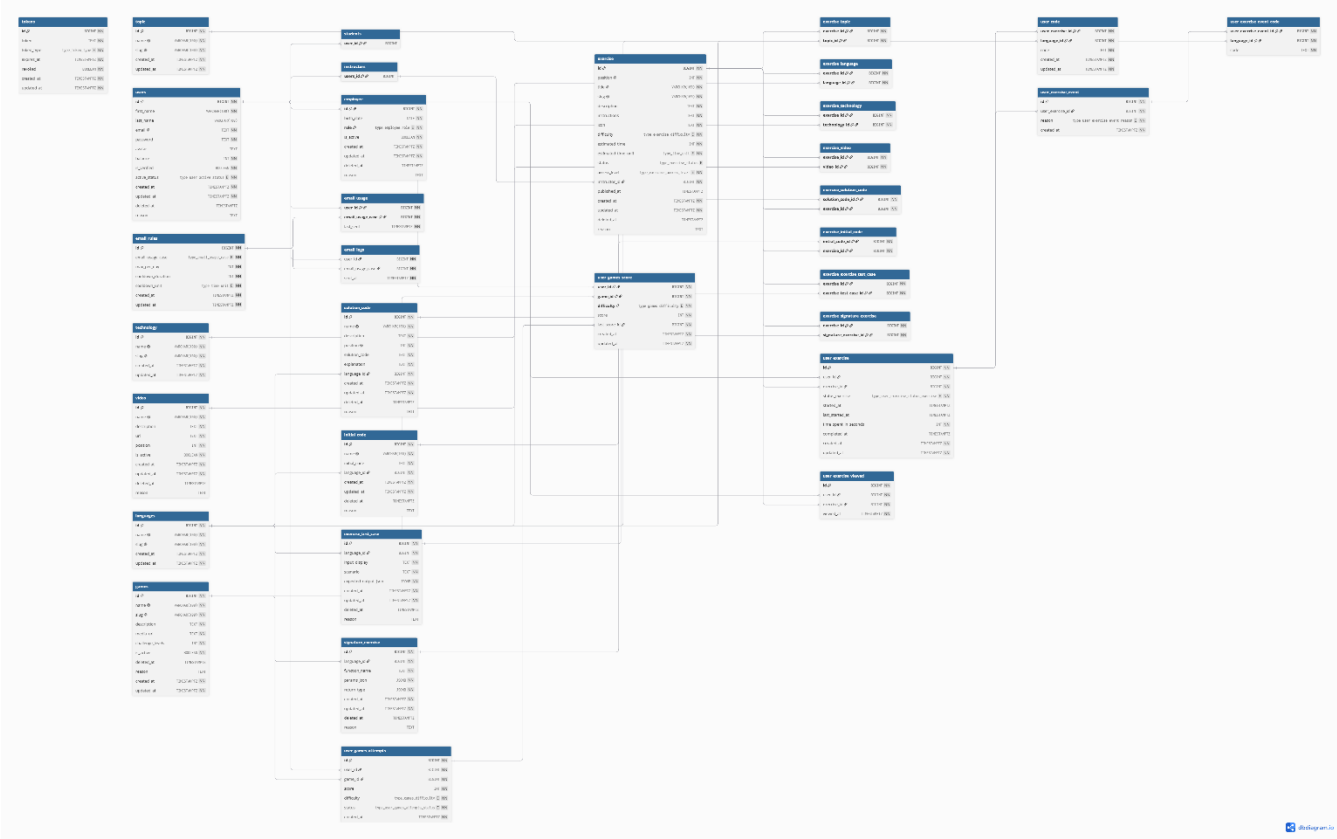
This project is about a relational database system using SQL. The goal is to create and design a database that can store, retrieve, and manage data with real world applications. The chosen for this project is **coding learning platform**, which is website where users can learn programming language depending on their desire and solving coding exercises.

In this platform users can register, browse learning paths, exercises, write and submit code, watch video lessons, and play learning games. Instructors can create exercises. The platform tracks users progress, store all versions of user code, and manages email notifications.

The main goal of this project are:

- Design clear and organized database schema using Entity-Relationship (ER) diagrams.
- Build the database with proper tables, primary keys, foreign keys, and constraints.
- Write SQL queries for CRUD operation.
- Show advanced queries such as joins and aggregations.
- Store all project files on GitHub.

The database is designed with several main groups of tables. Each group handles different purpose. Below is a description of each group and why this was designed in this way.

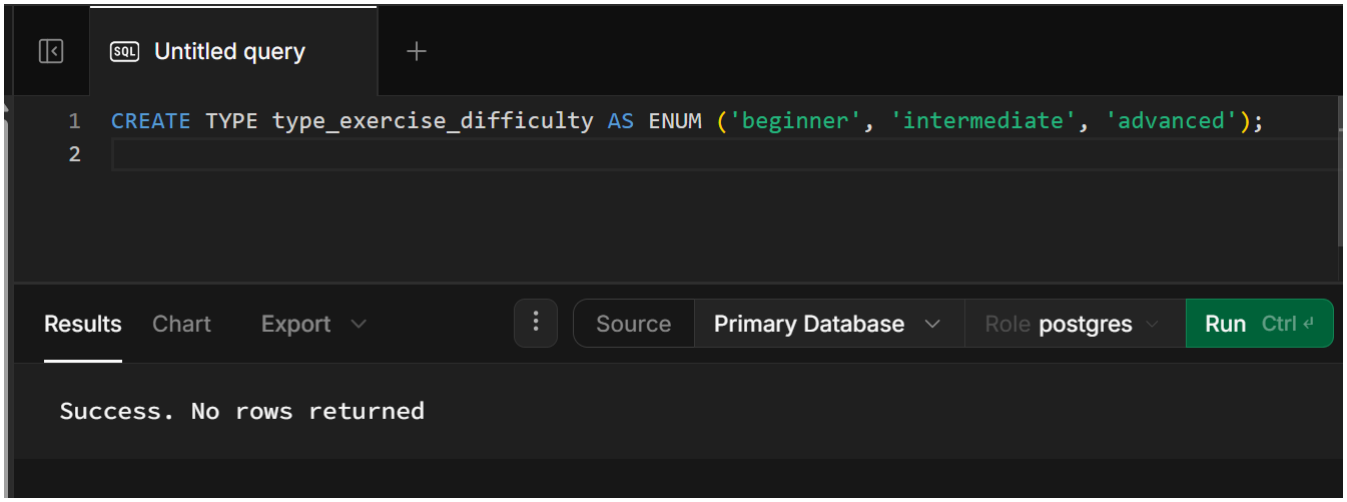


The ER diagram above shows all tables and how they have connection between. The main tables are:

- Users — stores all user accounts including students, instructors, and employees.
- Exercise — the main part of the table where coding exercises are stored.
- Languages & Topics & Technology — tables that categorize exercises.
- User Exercise — tracks which exercises a user has started or completed.
- Games — a mini-game feature where users can practice what they have learned so far.
- Tokens & Emails — handle authentication and email sending rules during registration.
- Video — stores lesson videos of exercises.

The database is built using PostgreSQL. The process follows step process: first creating enumerated types, then creating tables in the correct order so that foreign keys work in proper way.

Before creating tables, ENUM types were created to define the allowed values for certain columns. For example:



First tables create with no foreign keys (like users, languages, topic) and then moved to table that depend on them (like exercise, user_exercise). This order is important because PostgreSQL will give an error if a foreign key of table that does not exist yet.

```sql
CREATE TABLE exercise (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  position INT NOT NULL UNIQUE,
  title VARCHAR(100) NOT NULL UNIQUE,
  slug VARCHAR(100) NOT NULL UNIQUE,
  description TEXT NOT NULL,
  instructions TEXT NOT NULL,
  icon TEXT NOT NULL,
  difficulty type_exercise_difficulty NOT NULL,
  estimated_time INT NOT NULL,
  estimated_time_unit type_time_unit NOT NULL,
  status type_exercise_status NOT NULL,
  access_level type_exercise_access_level NOT NULL,
  instructor_id BIGINT NOT NULL REFERENCES instructors(users_id),
  published_at TIMESTAMPTZ DEFAULT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ DEFAULT NULL,
  reason TEXT DEFAULT NULL
);
```

```sql
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL CHECK (length(btrim(first_name)) > 0),
    last_name VARCHAR(100) DEFAULT NULL CHECK (length(btrim(last_name)) > 0),
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    avatar TEXT DEFAULT NULL,
    balance INT NOT NULL DEFAULT 0,
    is_verified BOOLEAN DEFAULT false,
    active_status type_user_active_status NOT NULL DEFAULT 'pending',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ DEFAULT NULL,
    reason TEXT DEFAULT NULL
);
```

Each table uses the following types of constraints:

- PRIMARY KEY — to uniquely identify each row.
- FOREIGN KEY — to link tables together and keep data consistent.
- NOT NULL — to make sure important fields always have a value.
- UNIQUE — to prevent duplication in values of columns like email or slug.
- DEFAULT — to set automatic values, for example created_at uses DEFAULT now().
- CHECK — to validate data, for example (length(btrim(first_name)) > 0).

Indexes were added to improve speed of queries. For instance, a unique index was created in the users table to make sure that only active users have unique email address. This method allows deleted users email to be reused later on.

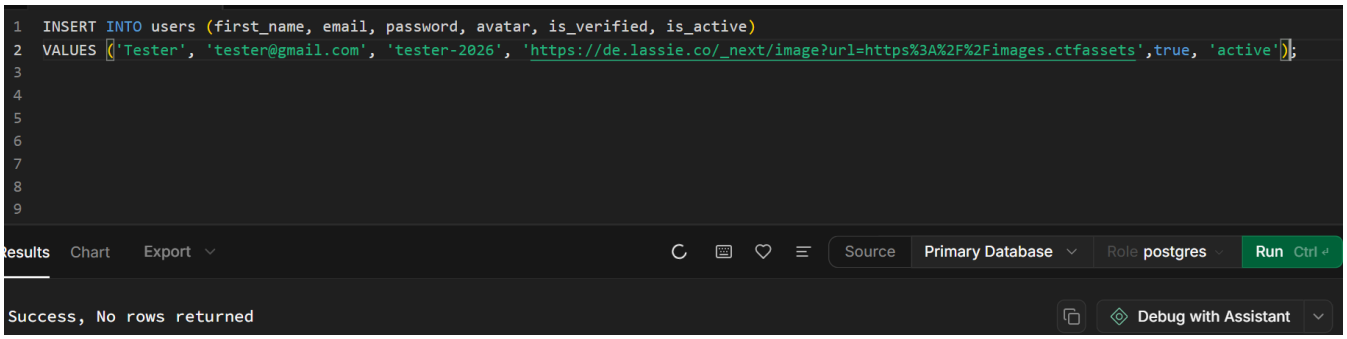
```sql
CREATE UNIQUE INDEX users_email_unique_active ON users (email) WHERE deleted_at IS NULL;
```

Results Chart Export Source Primary Database Role postgres Run Ctrl

Success. No rows returned

This section shows the results of running different SQL queries on the database. The queries demonstrate that the database works correctly and can handle different types of operations.

New records were added for the database using insert statements. For example, a new user was added to the users table, and new exercise was added to the exercise table.



The UPDATE statement was used to change existing records. For example, updating the status of an exercise from 'pending' to 'available' after it is reviewed.

The soft delete operating was used in most tables. Instead of using DELETE, the deleted_at column is updated with current timestamptz. This keeps the data while making it as deleted.

```
1  UPDATE exercise SET deleted_at = now(), reason = 'removed by admin' WHERE id = 1;
2  SELECT id, title, deleted_at,reason FROM exercise WHERE id = 18;
```

Results | Chart | Export | Source | Primary Database | Role postgres | Run selected Ctrl

id	title	deleted_at	reason
18	Repeated Number Triangle	NULL	removed by admin

More advanced queries were written in order to show useful information to the admin from the database. The query below counts how many exercises each topic has:

```
1  SELECT t.name AS topic, COUNT(et.exercise_id) AS total_exercises FROM topic t LEFT JOIN exercise_topic et ON t.
   id = et.topic_id GROUP BY t.name ORDER BY total_exercises DESC
```

Results | Chart | Export | Source | Primary Database | Role postgres | Run Ctrl

topic	total_exercises
Data Structures and Algorithms	3
Backend	0
Database	0
Frontend	0
Fullstack	0

Another advanced query shows the top users with the most completed exercises, but obviously there is one user – Tester.

```
1  SELECT u.first_name, u.email, COUNT(ue.id) AS completed FROM users u JOIN user_exercise ue ON u.id = ue.user_id
   WHERE ue.status_exercise = 'completed' GROUP BY u.id, u.first_name, u.email ORDER BY completed DESC LIMIT 5;
```

Results | Chart | Export | Source: Primary Database | Role: postgres | Run Ctrl

first_name	email	completed
Tester	tester@gmail.com	2

During the development process of this project, several problems were faced. Below are the main challenges and how they were solved step by step.

The first challenge was deciding the overall of structure of this project. Deciding how the table relations was one the hardest part of building database structure. Without thinking how many table and their relation with each other, later would make a lot of problems which is not easy to remake or fix relation of tables.

Second was deciding the correct order to create tables. If a table with a foreign key is created before the table it references, PostgreSQL gives an error. The solution for this was to plan the order carefully by first creating tables with no foreign keys, and then creating the dependent tables after.

Some relationships between tables were many-to-many. One exercise could be example as having many topics, and one topic can belong to many exercises. In a relational database this type cannot be stored directly. Creation of junction tables like exercise_topic that hold both foreign keys and usage of composite primary keys was the solution to the problem.

Choosing the right data for each column was also not easy part. For instance, using TIMESTAMPTZ instead of TIMESTAMPT was one the important because platform can be used by users in different time zones. Usage of JSONB for fields like function parameter in signature_exercise was chosen due to structure of parameter which can vary for different exercises.

This project successfully designed and implemented in a relational database app – supabase – for learning web app platform. The database has almost 30 tables organized in a clear groups: users, exercises, videos, games, and email management. All tables have own proper both primary and foreign keys, constraints, relationships, and follow normalization rules.

The database supports all main operations CRUD: inserting new, reading and filtering, and updating data as well as soft deleting data using column deleted_at with given reason.

Advanced queries such as JOIN across multiples tables, using LEFT, RIGHT, and INNER, and GROUP BY aggregations were also demonstrated.

In the coming future, this database could be improved in the following weeks:

- Add a payment and subscription table to support paid content.
- Add a comments or discussion table so users can discuss exercises.
- Splitting table in two tables wherever possible to normalize data
- Create database views for common reports such as user progress dashboards.
- Add more indexes to improve performance as the data grows.
- Implement stored procedures for complex operations like calculating scores.
- Implementing RLS and Object privileges for the tables to improve security of database

Overall, this project helped to develop overall understanding how real world databases are designed and built, and how SQL can be used to manage and query structured data effectively and securely.

References

PostgreSQL Documentation. (2024). PostgreSQL 16 Documentation. Retrieved from https://www.postgresql.org/docs/

dbdiagram.io. (2024). Database Relationship Diagram Tool. Retrieved from https://dbdiagram.io

Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.